tcga_estimate_purity.R

chu25

2025-08-26

```r
# ==============================================================================
# TIDYESTIMATE PURITY ANALYSIS WITH FILTERING
# ==============================================================================
# Purpose:
#   - Map Ensembl IDs → HGNC symbols using GTF.
#   - Harmonize symbols and filter to ESTIMATE gene signatures.
#   - Run tidyestimate scoring (stromal, immune, estimate).
#   - Normalize output, derive TumorPurity if missing.
#   - Filter samples based on tumor purity threshold (>= 0.6).
#   - Save results to CSV, update SummarizedExperiment colData,
#     and generate diagnostic correlation plots.
# ==============================================================================

options(repos = c(CRAN = "https://cloud.r-project.org"))

# -- Ensure dependencies are installed -----------------------------------------
required_pkgs <- c(
  "tidyestimate", "SummarizedExperiment", "dplyr",
  "ggplot2", "ggpubr", "data.table", "tibble"
)
for (pkg in required_pkgs) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    message(sprintf("[INFO] Installing missing package: %s", pkg))
    install.packages(pkg, repos = "https://cloud.r-project.org")
  }
}

# -- Load libraries ------------------------------------------------------------
suppressPackageStartupMessages({
  library(tidyestimate)
  library(SummarizedExperiment)
  library(dplyr)
  library(ggplot2)
  library(ggpubr)
  library(data.table)
  library(tibble)
})
```

```
## Warning: package 'MatrixGenerics' was built under R version 4.3.3
```

```
## Warning: package 'matrixStats' was built under R version 4.3.3
```

```
## Warning: package 'GenomicRanges' was built under R version 4.3.3

## Warning: package 'S4Vectors' was built under R version 4.3.3

## Warning: package 'IRanges' was built under R version 4.3.3

## Warning: package 'Biobase' was built under R version 4.3.3

## Warning: package 'dplyr' was built under R version 4.3.3

## Warning: package 'ggplot2' was built under R version 4.3.3

## Warning: package 'ggpubr' was built under R version 4.3.3

## Warning: package 'data.table' was built under R version 4.3.3

## Warning: package 'tibble' was built under R version 4.3.3
```

```
# -- Helper: Convert Ensembl IDs to HGNC symbols ------------------------------
ensure_hgnc_symbols <- function(M, gtf_path) {
  # Skip conversion if already symbols
  if (!any(grepl("^ENSG\\d", rownames(M), perl = TRUE))) {
    cat("[INFO] Row IDs look like gene symbols; skipping conversion.\n")
    return(M)
  }
  cat("[INFO] Mapping Ensembl IDs to HGNC using GTF:\n     ", gtf_path, "\n")

  # Remove version suffix from Ensembl IDs
  rn <- sub("\\.\\d+$", "", rownames(M))

  # Load GTF and extract gene_id  gene_name map
  gtf <- fread(
    gtf_path, sep = "\t", header = FALSE, quote = "",
    data.table = TRUE,
    col.names = c("seqname","source","feature","start","end",
                  "score","strand","frame","attr"),
    showProgress = FALSE
  )
  genes <- gtf[feature == "gene",
    .(gene_id   = sub('.*gene_id "([^"]+)".*', '\\1', attr),
      gene_name = sub('.*gene_name "([^"]+)".*', '\\1', attr))]
  genes <- genes[!duplicated(gene_id) & nzchar(gene_name)]
  map_sym <- setNames(genes$gene_name, genes$gene_id)

  # Map Ensembl IDs to symbols
  sym  <- unname(map_sym[rn])
  keep <- !is.na(sym) & nzchar(sym)
  cat(sprintf("[INFO] Mapped %d / %d Ensembl IDs (%.1f%%).\n",
              sum(keep), length(rn),
              100 * sum(keep) / length(rn)))
```

```
  # Keep mapped rows and collapse duplicates
  M_sym <- M[keep, , drop = FALSE]
  rownames(M_sym) <- toupper(sym[keep])
  before <- nrow(M_sym)
  M_sym <- rowsum(as.matrix(M_sym), group = rownames(M_sym),
                  reorder = TRUE)
  after <- nrow(M_sym)
  if (after < before) {
    cat(sprintf("[INFO] Collapsed duplicates: %d → %d.\n",
                before, after))
  }
  M_sym
}

# -- Helper: Select tidyestimate entry-point ---------------------------------
.get_estimate_fn <- function() {
  if ("estimate_score" %in% getNamespaceExports("tidyestimate")) {
    return(tidyestimate::estimate_score)
  }
  if ("estimate_scores" %in% getNamespaceExports("tidyestimate")) {
    return(tidyestimate::estimate_scores)
  }
  stop("No estimate_score(s) function in tidyestimate.")
}

# -- Helper: Normalize ESTIMATE output ---------------------------------------
normalize_estimate_columns <- function(x) {
  # Ensure data.frame with rownames as sample IDs
  if (is.data.frame(x) && "sample" %in% colnames(x)) {
    rn <- x$sample
    x <- as.data.frame(x)
    rownames(x) <- rn
    x$sample <- NULL
  } else if (is.matrix(x)) {
    x <- as.data.frame(x)
  }

  # Assign names if missing
  if (ncol(x) == 3 && (is.null(colnames(x)) ||
                       all(colnames(x) == ""))) {
    colnames(x) <- c("StromalScore", "ImmuneScore", "ESTIMATEScore")
    cat("[INFO] Assigned default column names.\n")
  }

  # Regex-based name remapping
  nml <- tolower(colnames(x))
  new_names <- colnames(x)

  if (any(grepl("strom", nml)))
    new_names[grepl("strom", nml)] <- "StromalScore"

  if (any(grepl("imm", nml)))
    new_names[grepl("imm", nml)] <- "ImmuneScore"
```

```
  if (any(grepl("est", nml)))
    new_names[grepl("est", nml)] <- "ESTIMATEScore"

  colnames(x) <- new_names

  # If ambiguous but exactly 3 cols, enforce default
  expected3 <- c("StromalScore", "ImmuneScore", "ESTIMATEScore")
  if (ncol(x) == 3 &&
      length(intersect(colnames(x), expected3)) < 3) {
    colnames(x) <- expected3
    cat("[INFO] Forced standard column names by position.\n")
  }

  # Derive TumorPurity if missing (Yoshihara formula)
  if (!"TumorPurity" %in% colnames(x) &&
      "ESTIMATEScore" %in% colnames(x)) {
    purity <- cos(0.6049872018 + 0.0001467884 * x$ESTIMATEScore)
    x$TumorPurity <- pmin(1, pmax(0, purity))
    cat("[INFO] Derived TumorPurity from ESTIMATEScore.\n")
  }

  # Final check
  expected <- c("StromalScore","ImmuneScore",
                "ESTIMATEScore","TumorPurity")
  miss <- setdiff(expected, colnames(x))
  if (length(miss)) {
    warning("[WARN] Missing columns: ", paste(miss, collapse=", "))
  } else {
    cat("[INFO] All expected ESTIMATE columns present.\n")
  }
  x
}

# -- Helper: Filter samples by tumor purity -----------------------------------
filter_by_purity <- function(tcga_se, purity_threshold = 0.6) {
  cat("[INFO] Filtering samples by tumor purity...\n")

  # Check if purity column exists
  if (!"purity" %in% colnames(colData(tcga_se))) {
    warning("[WARN] No 'purity' column found in colData. Skipping filtering.")
    return(tcga_se)
  }

  # Check purity range and distribution
  purity_vals <- colData(tcga_se)$purity
  cat("[INFO] Purity summary statistics:\n")
  print(summary(purity_vals))

  # Create logical filter: keep samples with purity >= threshold
  keep <- !is.na(purity_vals) & purity_vals >= purity_threshold

  # Apply filter
  tcga_se_filtered <- tcga_se[, keep]
```

```
  # Report filtering results
  n_kept <- sum(keep)
  n_total <- ncol(tcga_se)
  n_na <- sum(is.na(purity_vals))
  n_low_purity <- sum(!is.na(purity_vals) & purity_vals < purity_threshold)

  cat(sprintf("[INFO] Purity filtering results (threshold >= %.1f):\n", purity_threshold))
  cat(sprintf("  - Total samples: %d\n", n_total))
  cat(sprintf("  - Samples with NA purity: %d\n", n_na))
  cat(sprintf("  - Samples with low purity (< %.1f): %d\n", purity_threshold, n_low_purity))
  cat(sprintf("  - Samples kept (>= %.1f purity): %d (%.1f%%)\n",
              purity_threshold, n_kept, 100 * n_kept / n_total))

  return(tcga_se_filtered)
}

# -- MAIN FUNCTION -------------------------------------------------------------
run_tidyestimate_purity_analysis <- function(
  tcga_se_path = "/home/chu25/data/tcga/ALL_TCGA_STAR_Counts_SummarizedExperiment.rds",
  tcga_se_out  = "/home/chu25/data/tcga/ALL_TCGA_STAR_Counts_SummarizedExperiment_update.rds",
  tcga_se_filtered_out = "/home/chu25/data/tcga/ALL_TCGA_STAR_Counts_SummarizedExperiment_filtered.rds"
  output_dir   = "/home/chu25/dsmz/results/correlation",
  gtf_path     = "/home/chu25/data/GTF/Homo_sapiens.GRCh38.107.gtf",
  assay_name   = NULL,
  save_updated = TRUE,
  purity_threshold = 0.6,
  apply_purity_filter = TRUE
) {
  cat("[INFO] Starting tidyestimate purity analysis...\n")
  dir.create(output_dir, showWarnings = FALSE, recursive = TRUE)

  # Load TCGA SE
  if (!file.exists(tcga_se_path))
    stop("[ERROR] Missing file: ", tcga_se_path)
  tcga_se <- readRDS(tcga_se_path)

  # Extract assay
  expr_mat <- if (is.null(assay_name)) assay(tcga_se)
              else assay(tcga_se, assay_name)
  cat(sprintf("[INFO] Expression matrix: %d genes x %d samples\n",
              nrow(expr_mat), ncol(expr_mat)))

  # Map to HGNC
  expr_hgnc <- ensure_hgnc_symbols(expr_mat, gtf_path)

  # Filter to ESTIMATE signatures
  cat("[INFO] Filtering to ESTIMATE signature genes...\n")
  expr_sig <- suppressMessages(
    tidyestimate::filter_common_genes(
      as.data.frame(expr_hgnc), id = "hgnc_symbol",
      tidy = FALSE, tell_missing = TRUE, find_alias = TRUE
    )
  )
```

```r
# Run tidyestimate scoring
est_fn <- .get_estimate_fn()
cat("[INFO] Running tidyestimate scoring...\n")
est_scores <- est_fn(expr_sig, is_affymetrix = FALSE)

# Normalize columns, compute TumorPurity
est_scores <- normalize_estimate_columns(est_scores)
cat(sprintf("[INFO] Scores: %d samples x %d metrics\n",
            nrow(est_scores), ncol(est_scores)))

# Save CSV
out_csv <- file.path(output_dir, "tcga_tidyestimate_scores.csv")
write.csv(est_scores, out_csv, row.names = TRUE)
cat("[INFO] Saved scores to:", out_csv, "\n")

# Update SummarizedExperiment with purity
if (save_updated && "TumorPurity" %in% colnames(est_scores)) {
  common <- intersect(rownames(est_scores), colnames(tcga_se))
  colData(tcga_se)[, "purity"] <- NA_real_
  colData(tcga_se)[common, "purity"] <- est_scores[common, "TumorPurity"]
  saveRDS(tcga_se, tcga_se_out)
  cat(sprintf("[INFO] Updated colData with purity (%d/%d samples).\n",
              length(common), ncol(tcga_se)))

  # Apply purity filtering if requested
  if (apply_purity_filter) {
    tcga_se_filtered <- filter_by_purity(tcga_se, purity_threshold)
    saveRDS(tcga_se_filtered, tcga_se_filtered_out)
    cat(sprintf("[INFO] Saved filtered SummarizedExperiment to: %s\n", tcga_se_filtered_out))

    # Extract filtered count matrix for downstream analysis
    tcga_counts_filtered <- assay(tcga_se_filtered)
    cat(sprintf("[INFO] Filtered expression matrix: %d genes x %d samples\n",
                nrow(tcga_counts_filtered), ncol(tcga_counts_filtered)))
  }
}

# Purity correlation plots
if ("TumorPurity" %in% colnames(est_scores)) {
  cat("[INFO] Creating purity correlation plots...\n")
  df <- est_scores %>% as.data.frame() %>%
    rownames_to_column("Sample")

  p1 <- ggplot(df, aes(StromalScore, TumorPurity)) +
    geom_point(alpha=0.6) +
    geom_smooth(method="lm", se=FALSE) +
    ggpubr::stat_cor(method="spearman") +
    ggpubr::stat_cor(method="pearson", vjust=1.5) +
    theme_bw(12) +
    labs(title="Purity vs StromalScore")

  p2 <- ggplot(df, aes(ImmuneScore, TumorPurity)) +
    geom_point(alpha=0.6) +
```

```
      geom_smooth(method="lm", se=FALSE) +
      ggpubr::stat_cor(method="spearman") +
      ggpubr::stat_cor(method="pearson", vjust=1.5) +
      theme_bw(12) +
      labs(title="Purity vs ImmuneScore")

    # Add purity distribution histogram
    p3 <- ggplot(df, aes(TumorPurity)) +
      geom_histogram(bins = 50, fill = "steelblue", alpha = 0.7) +
      geom_vline(xintercept = purity_threshold, color = "red", linetype = "dashed", size = 1) +
      annotate("text", x = purity_threshold + 0.05, y = Inf,
               label = paste("Threshold =", purity_threshold),
               vjust = 1.2, hjust = 0, color = "red") +
      theme_bw(12) +
      labs(title = "Distribution of Tumor Purity",
           x = "Tumor Purity", y = "Count")

    # Combine plots
    pdf_file <- file.path(output_dir, "purity_correlation_plots.pdf")
    g <- ggpubr::ggarrange(p1, p2, p3, ncol = 2, nrow = 2)
    ggsave(pdf_file, g, width = 12, height = 10)
    cat("[INFO] Saved plots:", pdf_file, "\n")
  }

  # Prepare return object
  result <- list(
    scores = est_scores,
    output_csv = out_csv,
    tcga_se_updated = if(save_updated) tcga_se else NULL
  )

  if (apply_purity_filter && save_updated && "TumorPurity" %in% colnames(est_scores)) {
    result$tcga_se_filtered <- tcga_se_filtered
    result$tcga_counts_filtered <- tcga_counts_filtered
    result$filtered_output_path <- tcga_se_filtered_out
  }

  invisible(result)
}

# -- Run ----------------------------------------------------------------------
tryCatch({
  res <- run_tidyestimate_purity_analysis(
    apply_purity_filter = TRUE,
    purity_threshold = 0.6
  )
  cat("[INFO] tidyestimate analysis finished successfully.\n")

  # Print summary of results
  if (!is.null(res$tcga_se_filtered)) {
    cat(sprintf("[INFO] Filtered dataset ready for downstream analysis: %d genes x %d samples\n",
                nrow(res$tcga_counts_filtered), ncol(res$tcga_counts_filtered)))
  }
```

```
}, error = function(e) {
  cat("[ERROR] tidyestimate analysis failed:", e$message, "\n")
})
```

```
## [INFO] Starting tidyestimate purity analysis...
## [INFO] Expression matrix: 60660 genes x 11499 samples
## [INFO] Mapping Ensembl IDs to HGNC using GTF:
##       /home/chu25/data/GTF/Homo_sapiens.GRCh38.107.gtf
## [INFO] Mapped 60466 / 60660 Ensembl IDs (99.7%).
## [INFO] Collapsed duplicates: 60466 → 59267.
## [INFO] Filtering to ESTIMATE signature genes...
## [INFO] Running tidyestimate scoring...
```

```
## Number of stromal_signature genes in data: 141 (out of 141)
```

```
## Number of immune_signature genes in data: 141 (out of 141)
```

```
## [INFO] Derived TumorPurity from ESTIMATEScore.
## [INFO] All expected ESTIMATE columns present.
## [INFO] Scores: 11499 samples x 4 metrics
## [INFO] Saved scores to: /home/chu25/dsmz/results/correlation/tcga_tidyestimate_scores.csv
## [INFO] Updated colData with purity (11499/11499 samples).
## [INFO] Filtering samples by tumor purity...
## [INFO] Purity summary statistics:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.1855  0.7180  0.8348  0.8015  0.9154  0.9998
## [INFO] Purity filtering results (threshold >= 0.6):
##   - Total samples: 11499
##   - Samples with NA purity: 0
##   - Samples with low purity (< 0.6): 1227
##   - Samples kept (>= 0.6 purity): 10272 (89.3%)
## [INFO] Saved filtered SummarizedExperiment to: /home/chu25/data/tcga/ALL_TCGA_STAR_Counts_Summarized
## [INFO] Filtered expression matrix: 60660 genes x 10272 samples
## [INFO] Creating purity correlation plots...
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## [INFO] Saved plots: /home/chu25/dsmz/results/correlation/purity_correlation_plots.pdf
## [INFO] tidyestimate analysis finished successfully.
## [INFO] Filtered dataset ready for downstream analysis: 60660 genes x 10272 samples
```